

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

autoAI:recomendador
de vehículos mediante
LLM y web scraping

Curso: 2025/2026

Alumno/a:

Adrián Martínez Granados

Director/es:

Jesús Manuel Almendros Jiménez

ÍNDICE GENERAL

	Página
Resumen y Abstract	XII
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Descripción del problema	1
1.3. Objetivos del trabajo	1
1.3.1. Objetivo general	1
1.3.2. Objetivos específicos	1
1.4. Estructura de la memoria	1
2. Estado del arte	3
2.1. Buscadores y agregadores de coches existentes	3
2.2. Modelos de lenguaje y procesamiento de lenguaje natural	3
2.3. Web scraping para la obtención de datos	3
2.4. Tecnologías web modernas para aplicaciones interactivas	3
2.5. Conclusiones del análisis	3
3. Análisis de requisitos	5
3.1. Descripción general del sistema	5
3.2. Actores del sistema	5
3.3. Requisitos funcionales	5
3.4. Requisitos no funcionales	5
3.5. Casos de uso	5
4. Metodología y planificación	7
4.1. Metodología de desarrollo	7
4.2. Planificación temporal y fases del proyecto	7
4.3. Herramientas y entorno de trabajo	7
4.4. Control de versiones e integración continua	7
5. Diseño del sistema	9

5.1. Arquitectura general	9
5.2. Modelo de datos	11
5.2.1. Diseño de la base de datos	12
5.2.2. Normalización de los datos de los vehículos	14
5.3. Diseño del <i>backend</i> (API REST)	18
5.4. Diseño del pipeline de consulta con IA	19
5.5. Diseño de la interfaz de usuario	24
6. Implementación	31
6.1. Obtención de datos: scraping de km77.com	31
6.1.1. Diseño del spider con Scrapy	31
6.1.2. Automatización periódica del scraping	31
6.2. Proceso ETL y carga en la base de datos	31
6.3. Backend con FastAPI	31
6.3.1. Endpoints del catálogo y comparador	31
6.3.2. Modelos de datos y validación	31
6.4. Integración de la inteligencia artificial	31
6.4.1. Traducción de lenguaje natural a filtros estructurados	31
6.4.2. Bucle de <i>tool-calling</i> sobre datos reales	31
6.4.3. Generación de respuestas explicadas	31
6.5. Frontend con React	31
6.5.1. Interfaz conversacional	31
6.5.2. Catálogo, filtros y comparador	31
6.5.3. Favoritos e historial de consultas	31
6.6. Contenerización y despliegue con Docker	31
7. Pruebas y evaluación	33
7.1. Estrategia de pruebas	33
7.2. Pruebas del backend y del pipeline de IA	33
7.3. Evaluación de la calidad de los resultados	33
7.4. Discusión de los resultados	33
8. Conclusiones y trabajo futuro	35
8.1. Cumplimiento de los objetivos	35
8.2. Conclusiones	35
8.3. Líneas de trabajo futuro	35
BIBLIOGRAFÍA	35
A. Contenido de una entrada en .bib	39



B. Tipos de referencias

41

ÍNDICE DE FIGURAS

5.1. Diagrama de arquitectura general del sistema.	10
5.2. Esquema físico de la tabla «cars».	14
5.3. Diagrama de flujo del proceso de normalización e ingesta.	17
5.4. Diagrama de secuencia del pipeline de IA. Vista general.	22
5.5. Diagrama de secuencia del pipeline de IA. Bucle de tool-calling.	23
5.6. Diagrama de secuencia del pipeline de IA. Generación del envelope.	23
5.7. Diagrama de secuencia del pipeline de IA. Hidratación de imágenes.	24
5.8. Interfaz conversacional con respuesta de gráfica.	25
5.9. Interfaz conversacional con respuesta de tabla.	25
5.10. Interfaz conversacional con respuesta de imágenes.	26
5.11. Vista de catálogo con filtros.	26
5.12. Vista de favoritos.	27
5.13. Vista de historial.	28
5.14. Vista del panel de preferencias de accesibilidad.	29

ÍNDICE DE TABLAS

5.1. Bloques de atributos de la tabla <i>cars</i>	13
5.2. Vocabulario canónico del atributo <i>combustible</i>	15
5.3. <i>Endpoints</i> de la API REST del <i>backend</i>	18
5.4. Herramientas (<i>tools</i>) disponibles para el modelo de lenguaje.	20
A.1. Campos de una entrada en archivo de bibliografía <i>.bib</i>	40
B.1. Tipos posibles de elementos en bibliografía con <i>natbib</i>	42

LISTADOS

ABREVIATURAS

ESI	Escuela Superior de ingeniería
InSo	Ingeniería del Software
SBSE	Search based software engineering
TFG	Trabajo fin de grado
UAL	Universidad de Almería

RESUMEN Y ABSTRACT

En este archivo se incluirá tanto el resumen en castellano como su traducción al inglés. Los dos párrafos estarán ligeramente separados.

El propósito del trabajo en una o dos frases. El diseño y metodología utilizada, los resultados más significativos del trabajo realizado y un breve resumen de las conclusiones. Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG.

Put here the english translation Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG. Los dos párrafos estarán ligeramente separados.

1 INTRODUCCIÓN

1.1 CONTEXTO Y MOTIVACIÓN

1.2 DESCRIPCIÓN DEL PROBLEMA

1.3 OBJETIVOS DEL TRABAJO

1.3.1 Objetivo general

1.3.2 Objetivos específicos

1.4 ESTRUCTURA DE LA MEMORIA

2 ESTADO DEL ARTE

2.1 BUSCADORES Y AGREGADORES DE COCHES EXISTENTES

2.2 MODELOS DE LENGUAJE Y PROCESAMIENTO DE LENGUAJE NATURAL

2.3 WEB SCRAPING PARA LA OBTENCIÓN DE DATOS

2.4 TECNOLOGÍAS WEB MODERNAS PARA APLICACIONES INTERACTIVAS

2.5 CONCLUSIONES DEL ANÁLISIS

3 ANÁLISIS DE REQUISITOS

3.1 DESCRIPCIÓN GENERAL DEL SISTEMA

3.2 ACTORES DEL SISTEMA

3.3 REQUISITOS FUNCIONALES

3.4 REQUISITOS NO FUNCIONALES

3.5 CASOS DE USO

4 METODOLOGÍA Y PLANIFICACIÓN

4.1 METODOLOGÍA DE DESARROLLO

4.2 PLANIFICACIÓN TEMPORAL Y FASES DEL PROYECTO

4.3 HERRAMIENTAS Y ENTORNO DE TRABAJO

4.4 CONTROL DE VERSIONES E INTEGRACIÓN CONTINUA

5 DISEÑO DEL SISTEMA

En este capítulo se describe el diseño de *autoAI* en un nivel intermedio de abstracción: a partir de los requisitos recogidos en el capítulo anterior, se definen la arquitectura global del sistema, la estructura de los datos que lo sustentan, los servicios que lo componen y los flujos de información que los conectan. El objetivo es justificar las decisiones de diseño antes de descender, en el capítulo de implementación, al detalle del código. La exposición sigue un recorrido natural por las capas del sistema: primero la visión de conjunto y la forma en que se reparten las responsabilidades entre componentes; después el modelo de datos y su materialización en la base de datos; a continuación el diseño del *backend* y de su API REST; seguidamente el pipeline que traduce una consulta en lenguaje natural en una respuesta fundamentada en datos reales; y, por último, el diseño de la interfaz de usuario.

5.1 ARQUITECTURA GENERAL

autoAI se ha concebido como un sistema distribuido de cuatro **servicios independientes y desacoplados**, cada uno con una única responsabilidad y empaquetado en su propio contenedor. Esta separación responde directamente a la heterogeneidad de las tareas que el sistema debe afrontar: extraer datos de una fuente web externa, almacenarlos de forma estructurada, exponerlos a través de una API y razonar sobre ellos mediante un modelo de lenguaje, y presentarlos en una interfaz interactiva. Cada una de estas actividades tiene un ciclo de vida, unas dependencias tecnológicas y un patrón de ejecución distintos, por lo que aislarlas en componentes separados favorece el mantenimiento, permite escalarlas o sustituirlas por separado y evita que un fallo en una de ellas comprometa a las demás.

Los cuatro servicios son los siguientes:

- **Base de datos** (bd): una instancia de *PostgreSQL* que actúa como única fuente de verdad del sistema. Almacena el catálogo de vehículos ya normalizado y es el único componente con estado persistente.
- **Backend** (backend): un servicio construido con *FastAPI* (Python) que centraliza toda la lógica de negocio. Expone la API REST que consume la interfaz, contiene el proceso de normalización y carga de datos, y orquesta el pipeline de inteligencia artificial. Es el único componente que se comunica con la base de datos y con el proveedor externo del modelo de lenguaje.
- **Servicio de scraping** (scraper): un proceso basado en *Scrapy* que extrae periódicamente los datos de los vehículos del portal *km77.com*. No accede directamente a la

- **Frontend** (frontend): una aplicación web de página única (SPA) desarrollada con *React* y servida en producción por un servidor *nginx*. Implementa tanto el asistente conversacional como el catálogo navegable, y consume exclusivamente la API REST del *backend*.

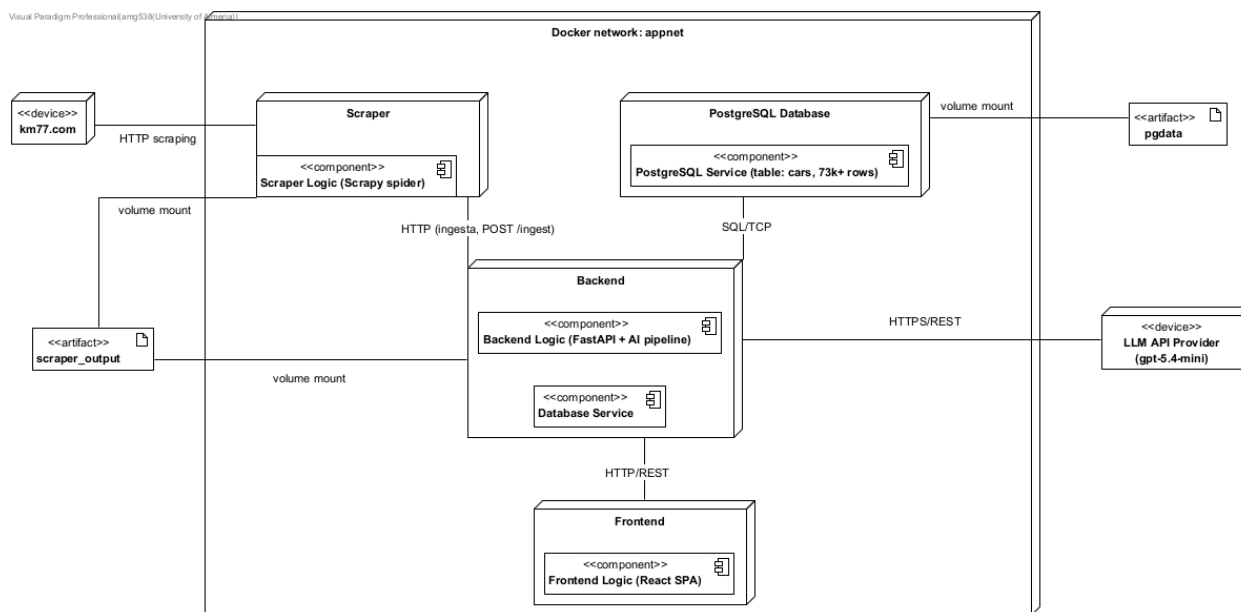


Figura 5.1: Diagrama de arquitectura general del sistema.

El elemento que articula esta arquitectura es la elección de un **punto de acceso único a los datos**: el *backend* es el único servicio autorizado a leer y escribir en la base de datos. El *frontend* nunca consulta la base de datos directamente, sino que pasa siempre por la API; y el *scraper*, aun produciendo los datos, tampoco los inserta por su cuenta, sino que delega su ingesta en el *backend* mediante una llamada HTTP. Esta decisión concentra en un solo lugar la validación, la normalización y el control de acceso, evita duplicar la lógica de persistencia y garantiza que ningún dato entre en la base de datos sin haber pasado antes por el mismo proceso de saneamiento.

Comunicación entre servicios

Los servicios se comunican mediante dos mecanismos complementarios. El principal es **HTTP/REST**: la interfaz consume los *endpoints* del *backend* para listar el catálogo, comparar vehículos y conversar con el asistente, y el *scraper* invoca el *endpoint* de ingesta cuando termina una extracción. El secundario es un **volumen de disco compartido** entre el *scraper* y el *backend*: el primero escribe en él el archivo con los datos extraídos y el segundo lo lee durante la ingesta. Este reparto evita que el *scraper* necesite conocer el esquema de la base de datos o las credenciales de acceso, y mantiene la frontera entre «producir datos crudos» y «validar y persistir datos» nítidamente trazada.



Todos los servicios residen en una misma red privada virtual y se descubren entre sí por su nombre lógico (bd, backend, etc.), sin necesidad de direcciones fijas. El arranque está ordenado según las dependencias reales: el *backend* no se inicia hasta que la base de datos responde a su comprobación de salud, y ni el *frontend* ni el *scraper* arrancan hasta que el *backend* está operativo. Con ello se elimina toda una clase de errores transitorios de arranque sin recurrir a esperas arbitrarias. Los detalles concretos de contenerización y orquestación se describen en el capítulo de implementación.

Patrón arquitectónico interno del *backend*

Dentro del *backend*, la organización del código sigue una **arquitectura en capas** que separa la exposición HTTP de la lógica de dominio y del acceso a datos. Se distinguen cuatro niveles:

- La capa de **API** (paquete `api`) define los *endpoints* REST, valida la entrada y traduce las peticiones HTTP en llamadas a la lógica de negocio. No contiene reglas de dominio ni SQL.
- La capa de **servicios** (paquete `services`) concentra la lógica de negocio: la normalización de los datos, el acceso a la base de datos (a través de un *repositorio*) y la integración con el modelo de lenguaje.
- La capa de **modelos** (paquete `models`) define las estructuras de datos con las que trabaja el sistema mediante *Pydantic*, lo que aporta validación automática y una frontera de tipos explícita.
- La capa **núcleo** (paquete `core`) reúne la configuración transversal y la gestión de la conexión a la base de datos.

Esta separación facilita las pruebas (cada capa puede probarse de forma aislada), localiza los cambios (una modificación en el esquema de la base de datos afecta al repositorio, no a los *endpoints*) y mantiene las dependencias dirigidas en un único sentido: la API depende de los servicios y estos de los modelos y el núcleo, nunca a la inversa.

5.2 MODELO DE DATOS

El modelo de datos de *autoAI* gira en torno a un único concepto central: la **versión de un vehículo**. El dominio de los automóviles es naturalmente jerárquico —una *marca* agrupa varios *modelos*, cada modelo se ofrece en distintos *submodelos* o generaciones, y cada submodelo incluye varias *versiones* o acabados con motorizaciones y equipamientos diferentes—, pero es la versión la que constituye la unidad mínima con significado para el usuario: es a ese nivel donde están definidos el precio, la potencia, el consumo o las dimensiones concretas. Por ello, la versión es la entidad sobre la que se construyen las búsquedas, las comparaciones y las recomendaciones.

Esta entidad se materializa en el sistema a través de dos representaciones distintas que conviene diferenciar desde el principio, porque reflejan dos momentos del ciclo de vida del dato:

- Una representación **cruda** (CocheScrap), que recoge los datos tal y como se extraen del portal de origen. Todos sus campos son cadenas de texto, porque así llegan del *scraping*: incluyen unidades («177 CV / 130 kW»), separadores de miles, símbolos de moneda, rangos de fechas y marcadores de dato ausente. Esta representación es fiel a la fuente, pero inservible para consultar o calcular.
- Una representación **normalizada** (Version), que es el modelo canónico del sistema. Cada atributo tiene ya su tipo correcto (numérico, fecha, texto), las unidades se han eliminado, los vocabularios se han unificado y la ausencia de dato se representa de forma explícita. Es la estructura que viaja por toda la aplicación: la que devuelve la base de datos, la que consume el pipeline de IA y la que recibe la interfaz.

El paso de la primera representación a la segunda es el proceso de normalización, descrito en la sección 5.2.2. Junto a estas dos estructuras existe una tercera familia de modelos, los **bloques de respuesta** del asistente (Block), que no describen vehículos sino el contenido estructurado que el modelo de lenguaje genera (texto, tablas, gráficas e imágenes); se detallan en la sección 5.4.

5.2.1 Diseño de la base de datos

La persistencia se resuelve con una base de datos relacional *PostgreSQL*. La decisión de diseño más característica es que el catálogo se almacena en una **única tabla**, *cars*, **deliberadamente desnormalizada**, en la que cada fila representa una versión completa de un vehículo con todos sus atributos. La jerarquía marca → modelo → submodelo → versión no se modela con tablas separadas y claves foráneas, sino que se aplanan en cuatro columnas de texto dentro de la misma fila (*marca*, *modelo*, *submodelo*, *nombre*).

Esta elección, que se aparta de la tercera forma normal que cabría esperar en un diseño relacional ortodoxo, está justificada por la naturaleza del problema:

- **La carga de trabajo es de solo lectura.** El usuario nunca edita el catálogo; este se regenera por completo desde la fuente. No existe, por tanto, el riesgo de anomalías de actualización que la normalización busca prevenir, que es precisamente su principal motivación.
- **El patrón de acceso dominante es el filtrado multicriterio sobre una sola entidad.** Casi todas las consultas (tanto del catálogo como de las herramientas de la IA) seleccionan versiones según combinaciones de atributos de la propia versión. Una tabla plana resuelve estas consultas sin una sola operación de *join*, lo que simplifica drásticamente el *repositorio* y mejora el rendimiento.
- **Reduce la complejidad accidental.** Para un catálogo de tamaño moderado y que cabe holgadamente en memoria, mantener un esquema en estrella o totalmente normalizado añadiría complejidad sin beneficio práctico apreciable.

Estructura de la tabla *cars*

La tabla *cars* reúne veintiséis columnas que se pueden agrupar conceptualmente en cinco bloques, recogidos en la tabla 5.1. La clave primaria es un identificador entero auto-

incremental (*id*), que es el que el sistema usa internamente para referirse a cada vehículo —en las comparaciones, en la recuperación de fichas y en los bloques de imagen generados por la IA—. Además, la columna *url* (la dirección de la ficha del vehículo en el portal de origen) está declarada **UNIQUE** y desempeña el papel de **clave de negocio**: identifica de forma estable una misma versión entre distintas ejecuciones del *scraping*, lo que permite reconocer si un vehículo ya existe y actualizarlo en lugar de duplicarlo.

Bloque		Columnas principales
Identificación		<i>id</i> (PK), <i>marca</i> , <i>modelo</i> , <i>submodelo</i> , <i>nombre</i> , <i>url</i> (única), <i>foto_url</i>
Vigencia		<i>fecha_inicio</i> , <i>fecha_fin</i> (inicio y fin de comercialización)
Precio		<i>precio</i> (en euros)
Carrocería y dimensiones	y	<i>carroceria</i> , <i>puertas</i> , <i>plazas</i> , <i>longitud</i> , <i>anchura</i> , <i>altura</i> , <i>capacidad_maletero</i>
Mecánica y prestaciones	y	<i>combustible</i> , <i>potencia</i> , <i>aceleracion</i> , <i>velocidad_maxima</i> , <i>peso</i> , <i>consumo_medio</i> , <i>traccion</i> , <i>transmision</i> , <i>numero_marchas</i> , <i>capacidad_deposito</i>

Tabla 5.1: Bloques de atributos de la tabla *cars*.

Los tipos de cada columna se han elegido para reflejar la semántica del dato y no su forma de origen. Las magnitudes físicas se almacenan como valores numéricos con la precisión adecuada (por ejemplo, el precio con dos decimales y las dimensiones con un decimal en milímetros), las fechas de comercialización como tipo **DATE**, los pequeños enteros (*puertas*, *plazas*, *número de marchas*) como enteros cortos, y los atributos categóricos (*combustible*, *carrocería*, *tracción*, *transmisión*) como cadenas que contienen siempre un valor del vocabulario canónico definido en la normalización.

Tratamiento de los datos ausentes

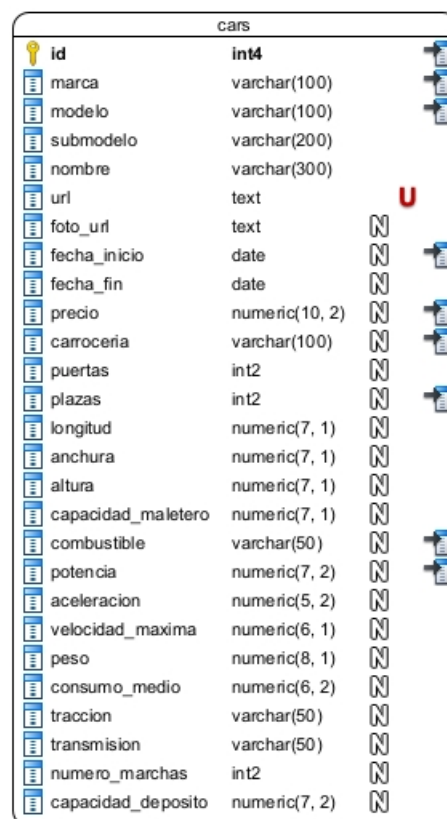
Una característica esencial del diseño es la admisión explícita de **valores nulos** en la mayoría de las columnas de atributos. La fuente de datos no es uniforme: hay vehículos sin maletero (como los *pick-up*), motorizaciones híbridas o eléctricas sin un número discreto de marchas, y campos que el portal sencillamente no registra. Modelar la ausencia con **NULL** —en lugar de con valores centinela como un cero o una cadena vacía— es fundamental porque permite distinguir «este coche pesa cero», que sería un dato erróneo, de «no se conoce el peso de este coche», que es la situación real. Esta distinción se propaga a todo el sistema: el *backend* ordena los resultados colocando siempre los valores nulos al final, y la interfaz los muestra como «dato no disponible» en lugar de inventar una cifra.

Un caso particular es la **vigencia**, modelada con el par de fechas *fecha_inicio* y *fecha_fin*. La convención es que una *fecha_fin* nula significa que la versión *sigue a la venta hoy*; es decir, el valor nulo no representa aquí un dato desconocido, sino un estado del mundo (vehículo aún en comercialización). Esta semántica es la que permite ofrecer al usuario, por defecto, el mercado actual y no versiones descatalogadas, y es un punto que tanto el *backend* como las instrucciones del modelo de lenguaje tratan de forma explícita.

Índices

Para sostener el rendimiento de las consultas más frecuentes sin recurrir a optimizaciones prematuras, el esquema define un conjunto reducido y deliberado de índices. Se indexan las columnas sobre las que más habitualmente se filtra (marca, modelo, precio, combustible, carrocería, potencia, plazas) y la columna que gobierna el orden por defecto del catálogo (fecha_inicio, ya que las novedades se muestran primero). Merece mención especial un **índice parcial** sobre las versiones vigentes —las que tienen fecha_fin nula—: al restringir el índice únicamente a las filas que cumplen esa condición, se acelera de forma muy eficiente la consulta «solo vehículos a la venta», que es una de las más recurrentes, ocupando además mucho menos espacio que un índice completo.

Visual Paradigm Professional(amg538(University of Almería))



cars	
id	int4
marca	varchar(100)
modelo	varchar(100)
submodelo	varchar(200)
nombre	varchar(300)
url	text
foto_url	text
fecha_inicio	date
fecha_fin	date
precio	numeric(10, 2)
carroceria	varchar(100)
puertas	int2
plazas	int2
longitud	numeric(7, 1)
anchura	numeric(7, 1)
altura	numeric(7, 1)
capacidad_maletero	numeric(7, 1)
combustible	varchar(50)
potencia	numeric(7, 2)
aceleracion	numeric(5, 2)
velocidad_maxima	numeric(6, 1)
peso	numeric(8, 1)
consumo_medio	numeric(6, 2)
traccion	varchar(50)
transmision	varchar(50)
numero_marchas	int2
capacidad_deposito	numeric(7, 2)

Figura 5.2: Esquema físico de la tabla «cars».

5.2.2 Normalización de los datos de los vehículos

La normalización es, posiblemente, la pieza de diseño más delicada del sistema. Los datos que llegan del *scraping* son fieles a la fuente pero heterogéneos y «sucios»: vienen como texto con unidades incrustadas, con formatos de número propios del español, con rangos de fechas escritos como cadenas y, sobre todo, con vocabularios inconsistentes y datos implícitos. El proceso de normalización tiene como misión convertir esa materia prima en el modelo canónico *Version*, de forma que el resto del sistema —y en particular el modelo de

lenguaje— pueda razonar sobre datos limpios, tipados y con un vocabulario cerrado y predecible. Se distinguen tres tareas: el saneamiento de los valores escalares, la unificación de los vocabularios categóricos y la persistencia idempotente.

Saneamiento de valores escalares

La parte más mecánica consiste en extraer de cada cadena el valor numérico o la fecha que contiene, descartando todo lo accesorio. Se aplican reglas específicas por tipo de campo: de «177 CV / 130 kW» se conserva únicamente la cifra en caballos; de «4.165 mm» se obtiene el valor en milímetros eliminando el separador de miles; de «7,2 1/100 km» se extrae el número cambiando la coma decimal por punto; y los rangos de fechas del tipo «(03/2021 - 09/2023)» se descomponen en las dos fechas que delimitan la vigencia. Un principio transversal gobierna todo este proceso: ante un valor ausente o un marcador de dato faltante (el portal usa un guion), la conversión **nunca falla ni inventa**, sino que produce un valor nulo. De este modo, una sola versión con un campo problemático no aborta la importación del lote completo.

Unificación de vocabularios categóricos

El reto más interesante del diseño no son los números, sino los atributos categóricos, y muy especialmente el **tipo de combustible**. El portal de origen no clasifica los vehículos por su grado de electrificación: en su campo de combustible solo indica la fuente del motor de combustión (gasolina, gasóleo o combinaciones con gas), y deja ese campo vacío para los eléctricos puros. La información de si un coche es híbrido suave, híbrido autorrecargable o híbrido enchufable no está en ese campo, sino *implícita* en el nombre comercial de la versión.

El diseño aborda este problema definiendo un **vocabulario canónico cerrado de nueve valores** que codifica, en un único atributo, tanto la fuente fósil como el nivel de electrificación (tabla 5.2). El proceso de normalización del combustible determina, por un lado, la base fósil a partir del campo de origen y, por otro, el nivel de electrificación analizando el texto del nombre y el submodelo con expresiones regulares que reconocen los distintos términos comerciales, aplicados en orden de mayor a menor especificidad (enchufable antes que autorrecargable, y este antes que suave) para evitar clasificaciones erróneas. Es importante señalar que este vocabulario es el mismo que después se ofrece al modelo de lenguaje como conjunto cerrado de valores válidos: unificar aquí el vocabulario es lo que permite que la IA filtre con precisión más adelante.

Valor canónico	Significado
gasolina	Gasolina pura, sin electrificar
gasoleo	Diésel puro, sin electrificar
gas	<i>Bi-fuel/flex</i> : gasolina con GLP, gas natural o etanol
electrico	100 % eléctrico (BEV)
mhev_gasolina	Microhíbrido (<i>mild hybrid</i> 48 V) de gasolina
mhev_gasoleo	Microhíbrido (<i>mild hybrid</i> 48 V) de diésel
hev_gasolina	Híbrido autorrecargable (<i>full hybrid</i>) de gasolina
phev_gasolina	Híbrido enchufable (PHEV) de gasolina
phev_gasoleo	Híbrido enchufable (PHEV) de diésel

Tabla 5.2: Vocabulario canónico del atributo combustible.

Un tratamiento análogo, aunque más sencillo, recibe el **tipo de carrocería**. El portal emplea sus propias denominaciones (Turismo, SUV/Todoterreno, Turismo familiar, Monovolumen, Descapotable, Pick Up, etc.), que se traducen a un vocabulario canónico de nueve categorías (berlina, suv, compacto, familiar, coupe, cabrio, monovolumen, pickup, furgoneta) mediante una combinación de correspondencias exactas y reglas por subcadena que absorben además sinónimos habituales (*crossover*, *ranchera*, MPV...). El resto de atributos categóricos (tracción y transmisión) se canonizan simplemente pasándolos a minúsculas.

Idempotencia

Una propiedad de diseño exigida a las funciones de normalización de vocabulario es la **idempotencia**: aplicar la normalización a un valor ya canónico debe devolver ese mismo valor sin alterarlo. Esto es importante porque el catálogo se reimporta periódicamente, y debe poder reprocesarse un dato ya normalizado sin que se corrompa (por ejemplo, sin que un valor ya en forma canónica se vuelva a transformar y deje de ser válido). La idempotencia hace que el proceso de ingesta sea seguro de repetir, lo que simplifica enormemente la operación del sistema.

Persistencia idempotente y tolerante a fallos

La inserción en la base de datos se ha diseñado también con dos garantías. La primera es la idempotencia a nivel de fila: la sentencia de inserción utiliza la cláusula `ON CONFLICT` sobre la columna `url` de modo que, si una versión ya existe (identificada por su clave de negocio), sus datos se *actualizan* en lugar de provocar un error o un duplicado. Así, el catálogo converge siempre al estado de la última extracción, tanto si se trata de una carga inicial como de una actualización.

La segunda garantía es la **tolerancia a fallos parciales**. Cada versión se inserta dentro de su propio punto de guardado (*savepoint*) anidado, de forma que si una fila concreta provoca un error en la base de datos, solo se revierte esa fila y las demás se conservan. El proceso, además, no se limita a guardar: devuelve un informe detallado del resultado de la importación —cuántos registros se leyeron, cuántos se validaron correctamente, cuántos se normalizaron y cuántos se persistieron, junto con la lista de los que fallaron y por qué—. Este enfoque prioriza la *robustez* sobre la atomicidad estricta: dado que los datos provienen de un *scraping* de una web externa y, por su naturaleza, son imperfectos, es preferible cargar todo lo que sea correcto e informar de lo que no, antes que rechazar un lote completo por unas pocas filas defectuosas.

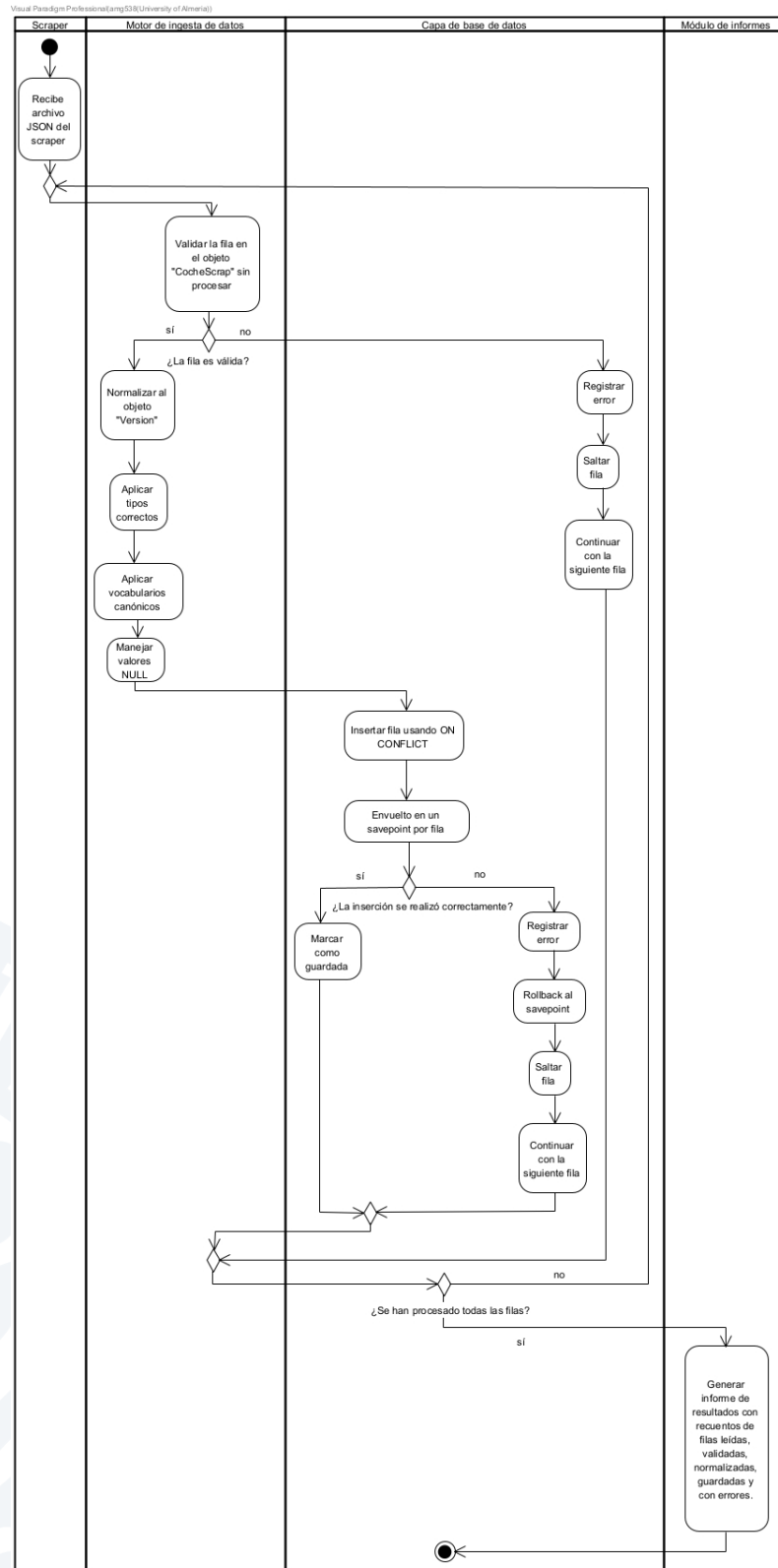


Figura 5.3: Diagrama de flujo del proceso de normalización e ingesta.

5.3 DISEÑO DEL *backend* (API REST)

El *backend* es el núcleo lógico del sistema y expone sus capacidades a través de una **API REST**. Su diseño persigue tres metas: ofrecer a la interfaz un contrato de datos claro y tipado, mantener la lógica de acceso a la base de datos centralizada y segura, y servir de orquestador para el pipeline de inteligencia artificial. La API se organiza en torno a dos grandes familias de *endpoints* —los del catálogo y comparador, de naturaleza determinista, y el del asistente conversacional, de naturaleza generativa—, más un par de *endpoints* de servicio. La tabla 5.3 resume el conjunto.

Método	Ruta	Función
GET	/api/cars	Lista de versiones filtrada y paginada (catálogo)
GET	/api/cars/{id}	Ficha completa de una versión
POST	/api/compare	Fichas de varias versiones para comparar
GET	/api/filters/meta	Valores distintos disponibles para los filtros
POST	/chat	Consulta en lenguaje natural al asistente
POST	/ingest	Ingesta de un archivo del <i>scraper</i> (interno)
GET	/health	Comprobación de estado del servicio

Tabla 5.3: *Endpoints* de la API REST del *backend*.

Endpoints del catálogo y comparador

Los *endpoints* del catálogo dan servicio a la navegación clásica de la interfaz. El de listado acepta como parámetros de consulta un conjunto de filtros (marca, modelo, combustible, carrocería, tracción, transmisión y rangos de precio, potencia y plazas) junto con la paginación, y devuelve la lista de versiones que los cumplen. El de ficha individual recupera un vehículo por su identificador, y el de comparación recibe una lista de identificadores y devuelve sus fichas completas. Un *endpoint* auxiliar, el de metadatos de filtros, devuelve los valores realmente presentes en la base de datos para cada atributo categórico, lo que permite a la interfaz poblar dinámicamente sus desplegables sin codificar los valores manualmente.

Una decisión de diseño relevante es que las respuestas de estos *endpoints* se ajustan exactamente al modelo canónico *Version*. El esquema de la respuesta se declara de forma explícita, lo que aporta tres beneficios: la documentación interactiva de la API se genera automáticamente, la validación de la salida está garantizada por el *framework*, y la interfaz puede definir un tipo «espejo» que refleja exactamente esa estructura, de modo que el contrato de datos entre *frontend* y *backend* queda fijado y verificable en ambos extremos.

El repositorio y la construcción segura de consultas

Toda la interacción con la base de datos se concentra en un único módulo, el **repositorio de vehículos**, que actúa como frontera entre la lógica de negocio y el SQL. Este patrón evita que las sentencias SQL se dispersen por el código y permite construir las consultas de forma

dinámica a partir de un diccionario de filtros: el repositorio traduce cada filtro presente en una condición de la cláusula `WHERE` y omite los ausentes, soportando tanto igualdades simples como filtros de pertenencia a un conjunto (por ejemplo, varios tipos de combustible a la vez) y rangos numéricos.

El aspecto más importante de este módulo es su **tratamiento de la seguridad**. Todos los valores aportados por el usuario se pasan a la consulta como **parámetros vinculados**, nunca por interpolación de cadenas, lo que elimina por construcción el riesgo de inyección de SQL. En los pocos casos en que es inevitable insertar un identificador en la propia estructura de la consulta —por ejemplo, el nombre de la columna por la que ordenar o agregar—, ese identificador se valida previamente contra una **lista blanca** de valores permitidos: el nombre de ordenación se busca en un diccionario cerrado de órdenes válidos, y los campos y operaciones de agregación se comprueban contra conjuntos predefinidos. Cualquier valor fuera de esas listas se rechaza antes de construir la consulta. De este modo se concilian la flexibilidad de las consultas dinámicas con la imposibilidad de que un valor arbitrario llegue al motor de la base de datos.

Endpoints de servicio y configuración

Completan la API dos *endpoints* de servicio. El de comprobación de estado permite que la orquestación verifique que el *backend* está vivo antes de arrancar los servicios que dependen de él. El de ingesta es el que invoca el *scraper* para solicitar la carga de un archivo; por seguridad, su diseño restringe el acceso al directorio compartido y rechaza cualquier intento de acceder a rutas fuera de él. Toda la configuración sensible del *backend* (la cadena de conexión a la base de datos, la clave del proveedor de IA, los orígenes permitidos para las peticiones de la interfaz) se externaliza mediante variables de entorno y se valida al arranque, de forma que el código no contiene ningún secreto y el comportamiento puede ajustarse entre entornos sin recompilar. Adicionalmente, el *backend* incorpora sendos mecanismos opcionales de limitación de frecuencia de peticiones y de autenticación por clave para los *endpoints* sensibles, desactivados por defecto y activables por configuración.

5.4 DISEÑO DEL PIPELINE DE CONSULTA CON IA

El pipeline de inteligencia artificial es el componente diferencial de *autoAI* y el que materializa su objetivo principal: permitir que un usuario exprese sus necesidades en **lenguaje natural** y reciba una recomendación **fundamentada en datos reales**. El reto de diseño central es conciliar dos mundos en tensión: la flexibilidad y la fluidez de un modelo de lenguaje, que puede interpretar peticiones ambiguas y redactar respuestas naturales, con el **rigor factual** de una base de datos, que es la única fuente legítima de cifras. La estrategia adoptada para resolver esta tensión es no permitir nunca que el modelo invente datos, sino obligarlo a obtenerlos de la base de datos a través de un conjunto acotado de herramientas, en un patrón conocido como *tool-calling*.

De lenguaje natural a filtros estructurados: las herramientas

En lugar de pedir al modelo que genere SQL —lo que sería potente pero arriesgado—, el diseño le ofrece un **catálogo cerrado de cuatro herramientas** de alto nivel que encapsulan las operaciones que el sistema sabe hacer con seguridad (tabla 5.4). La tarea del modelo se

reduce entonces a traducir la intención del usuario en una llamada a la herramienta adecuada con los argumentos correctos; es decir, a convertir el lenguaje natural en un conjunto de **filtros estructurados**.

Herramienta	Función
buscar_coches	Busca versiones que cumplen unos filtros, con orden y límite opcionales
obtener_coche	Devuelve la ficha completa de un vehículo por su identificador
comparar	Devuelve las fichas de varios vehículos para compararlos
agregar	Calcula una agregación (recuento, media, mínimo, máximo) sobre un campo, opcionalmente agrupada; es la fuente de las gráficas

Tabla 5.4: Herramientas (*tools*) disponibles para el modelo de lenguaje.

La clave del diseño está en la **especificación de los argumentos** de estas herramientas. El esquema de filtros que se entrega al modelo no es una lista libre, sino un contrato estricto: enumera explícitamente los atributos por los que se puede filtrar y, para los categóricos, restringe los valores admisibles al vocabulario canónico definido en la normalización. Es aquí donde la inversión hecha en la unificación de vocabularios rinde sus frutos: como el campo de combustible solo admite los nueve valores canónicos, el modelo no puede pedir un «híbrido» genérico ni un «diésel» con otra grafía, sino que debe traducir la intención del usuario a los valores exactos que existen en la base de datos. El propio esquema incluye, a modo de instrucciones, las equivalencias entre el lenguaje coloquial y el vocabulario canónico, lo que guía al modelo en esa traducción.

El bucle de *tool-calling* sobre datos reales

El corazón del pipeline es un **bucle de razonamiento** que alterna entre el modelo de lenguaje y la ejecución de herramientas. Su funcionamiento es el siguiente: se envía al modelo la conversación junto con la descripción de las herramientas; si el modelo decide llamar a una o varias, el *backend* las ejecuta realmente contra la base de datos y le devuelve los resultados; el modelo puede entonces razonar sobre esos datos y decidir si necesita más información (refinar la búsqueda, comparar, agregar) o si ya está en condiciones de responder. El ciclo se repite hasta que el modelo deja de pedir herramientas, con un **límite máximo de iteraciones** que actúa como salvaguarda para evitar bucles infinitos y acotar el coste de cada consulta.

Este diseño garantiza la invariante fundamental del sistema: **toda cifra que aparece en una respuesta procede de una consulta real a la base de datos**. El modelo aporta la interpretación, la estrategia de búsqueda y la redacción, pero los datos los pone siempre la base de datos. Sobre esta invariante se articula además un comportamiento experto, codificado en las instrucciones de sistema que acompañan a cada consulta: el modelo recibe directrices sobre cómo interpretar peticiones vagas («algo para la familia», «un coche para ciudad») traduciéndolas a filtros concretos, cómo priorizar por defecto el mercado actual, cómo actuar cuando el usuario filtra por un atributo que el sistema no soporta (explicarlo y ofrecer igualmente una búsqueda aproximada) y cómo tratar casos delicados como las unidades de consumo de los híbridos enchufables.

Un matiz de diseño interesante distingue entre **consultas objetivas y consultas exploratorias**. Cuando el usuario pide algo con un criterio inequívoco («el más barato», «el más potente»), la búsqueda devuelve exactamente ese resultado de forma determinista. Pero cuando la petición es cualitativa y sin un orden objetivo («un coche bonito para ciudad»), el sistema recupera un conjunto de candidatos más amplio y selecciona entre ellos con un muestreo sesgado hacia los más relevantes, de manera que dos consultas parecidas no devuelvan siempre exactamente los mismos vehículos. Con ello se evita que el asistente resulte repetitivo sin que por ello se muestren opciones de peor calidad.

Generación de respuestas estructuradas y explicadas

Una recomendación de coches no se expresa bien solo con texto: a menudo requiere una tabla comparativa, una gráfica de precios o la fotografía del vehículo. Por eso, la respuesta del asistente no es una cadena de texto plano, sino un documento estructurado compuesto por una secuencia de **bloques tipados**. Se definen cuatro tipos de bloque: texto (en formato *Markdown*), tabla, gráfica (de barras o radar) e imagen. El modelo, una vez ha reunido los datos a través de las herramientas, emite una respuesta final que se ajusta de forma estricta a este esquema de bloques, lo que permite a la interfaz renderizar cada uno con el componente adecuado.

Este enfoque exige una **segunda fase** en el pipeline, separada del bucle de herramientas: una vez recopilados todos los datos, se realiza una última llamada al modelo solicitándole que produzca el documento de bloques en un formato verificable. El *backend* valida que la respuesta cumple el esquema y, si no lo cumple, reintenta una vez indicándole el error; si tampoco así se obtiene una respuesta válida, se entrega un bloque de texto de reserva en lugar de propagar un fallo a la interfaz.

Dos decisiones de diseño refuerzan en esta fase la integridad de los datos. La primera afecta a las **imágenes**: para evitar que el modelo invente o altere direcciones de fotografías, se le prohíbe emitir URLs; en su lugar, un bloque de imagen solo contiene el identificador del vehículo, y es el *backend* quien, en un paso posterior de «hidratación», recupera de la base de datos la URL real de la foto y la inserta en el bloque (descartándolo si el vehículo no existe o no tiene imagen). La segunda afecta a las **gráficas**: estas se alimentan exclusivamente de la salida de la herramienta de agregación, de modo que las barras de un gráfico reflejan siempre valores calculados por la base de datos y no estimaciones del modelo; las etiquetas, unidades y títulos son metadatos de presentación que nunca modifican las cifras subyacentes. En conjunto, estas medidas trasladan la invariante de «no inventar datos» también a los elementos visuales de la respuesta.

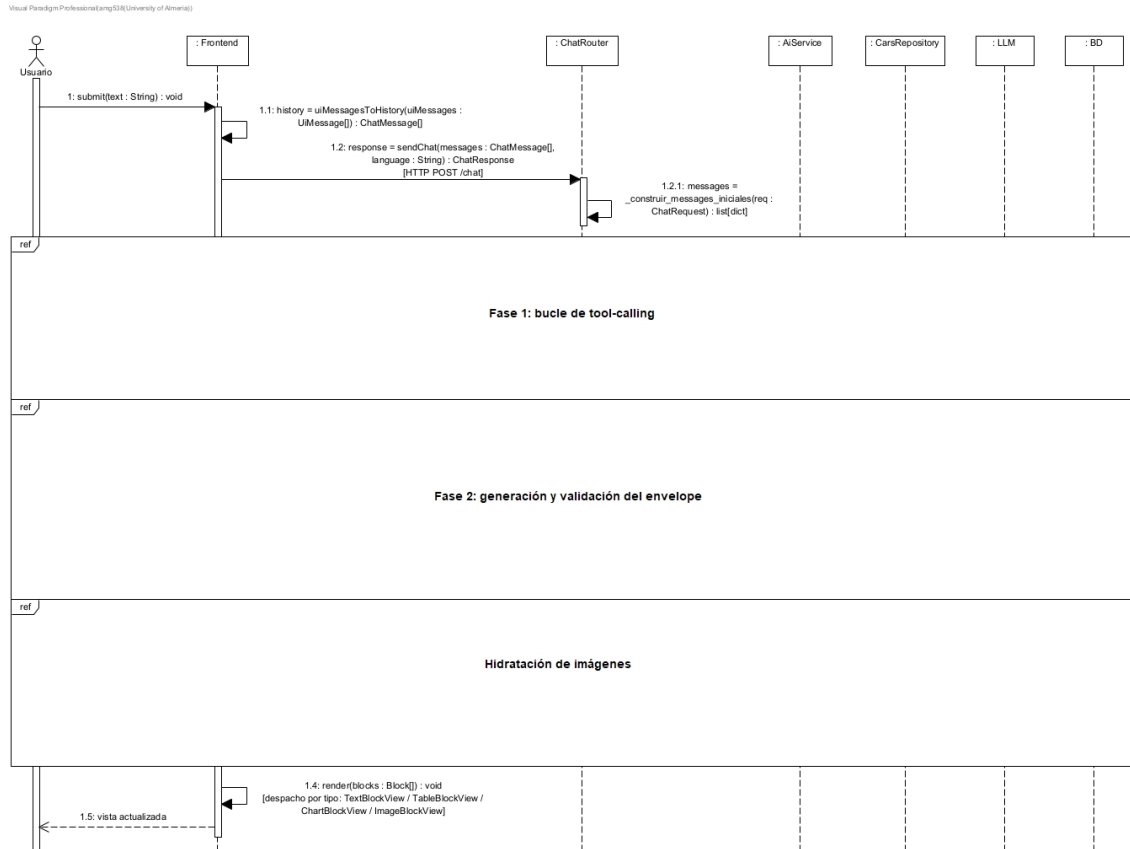


Figura 5.4: Diagrama de secuencia del pipeline de IA. Vista general.

5.4. DISEÑO DEL PIPELINE DE CONSULTA CON IA

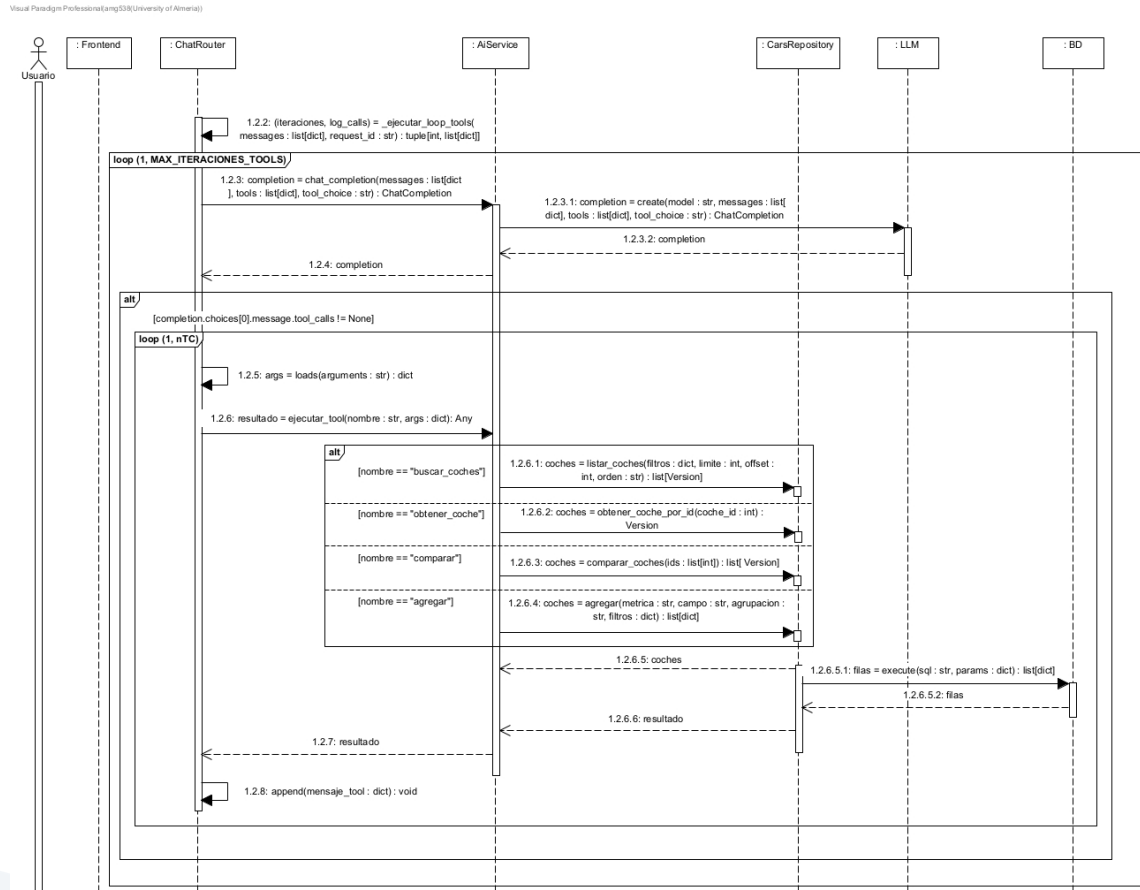


Figura 5.5: Diagrama de secuencia del pipeline de IA. Bucle de tool-calling.

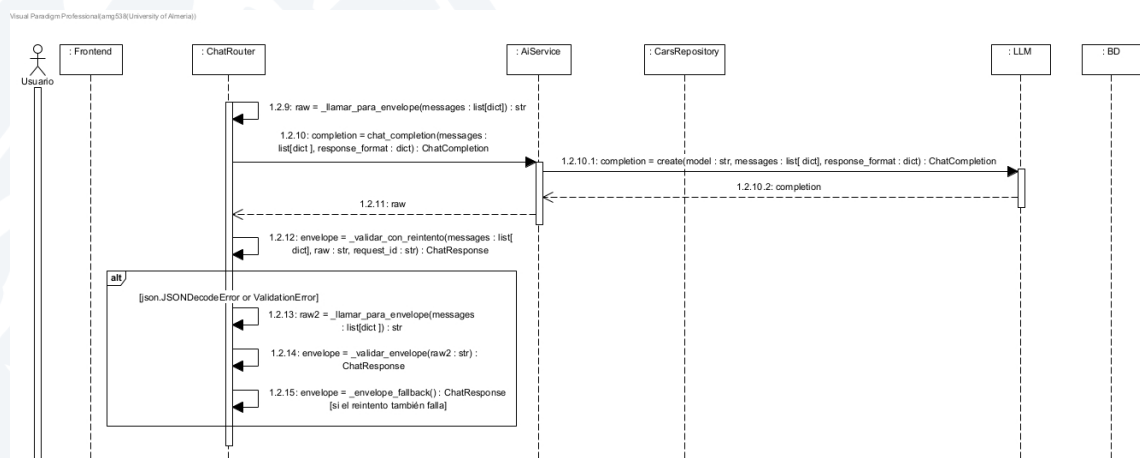


Figura 5.6: Diagrama de secuencia del pipeline de IA. Generación del envelope.

Visual Paradigm Professionals(amg538/University of Almería)

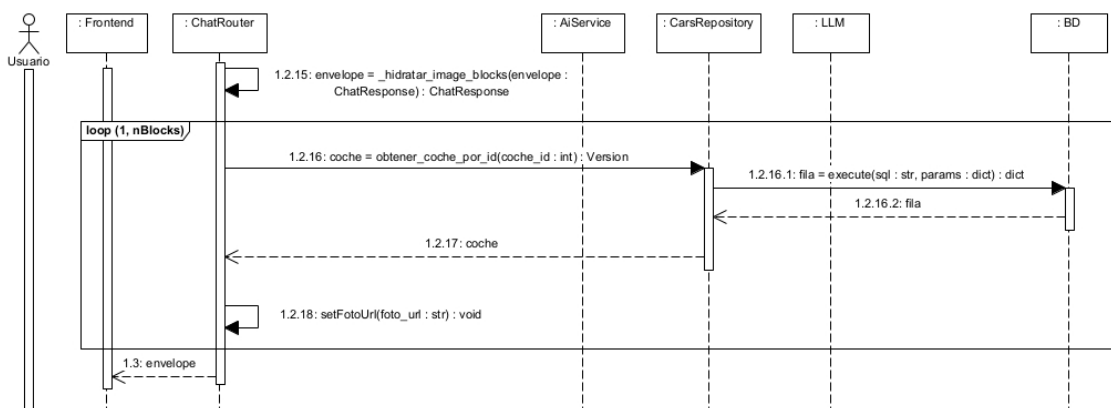


Figura 5.7: Diagrama de secuencia del pipeline de IA. Hidratación de imágenes.

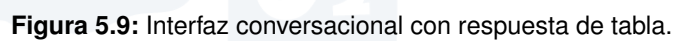
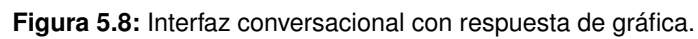
5.5 DISEÑO DE LA INTERFAZ DE USUARIO

La interfaz de usuario se ha diseñado como una **aplicación web de página única** que ofrece una experiencia fluida y sin recargas, organizada en torno a una barra lateral de navegación que da acceso a sus cuatro áreas funcionales: el **asistente conversacional**, el **catálogo**, el **historial** de conversaciones y los **favoritos**. El criterio rector del diseño es ofrecer dos modos de exploración complementarios sobre el mismo catálogo: uno *conversacional*, guiado por la IA, para el usuario que prefiere expresar sus necesidades con sus propias palabras; y otro *estructurado*, mediante filtros explícitos, para quien prefiere acotar la búsqueda por su cuenta.

Interfaz conversacional

El asistente es la vista principal y se presenta inicialmente con una pantalla de bienvenida que invita a escribir la primera consulta. A partir de ahí, la interacción adopta la forma familiar de una conversación, en la que se alternan los mensajes del usuario y las respuestas del asistente. El reto de diseño específico de esta vista es la **renderización de las respuestas estructuradas**: como la respuesta del *backend* es una secuencia de bloques tipados, la interfaz recorre esos bloques y delega cada uno en un componente especializado —el texto se interpreta como *Markdown*, las tablas se pintan como tablas con desplazamiento horizontal, las gráficas se dibujan con una biblioteca de visualización y las imágenes se muestran como fichas con una acción para añadir el vehículo a favoritos—. Este mecanismo de despacho por tipo de bloque es el que permite que una misma respuesta combine, por ejemplo, un párrafo introductorio, una tabla comparativa y una gráfica de precios.

El diseño cuida varios detalles de la experiencia conversacional: mientras el *backend* procesa la consulta se muestra un indicador de escritura, la vista se desplaza automáticamente al último mensaje, los errores de comunicación se presentan como un mensaje del asistente en lugar de como un fallo abrupto, y junto al campo de entrada figura de forma permanente un aviso de que la IA puede cometer errores, en línea con un uso responsable de la herramienta.



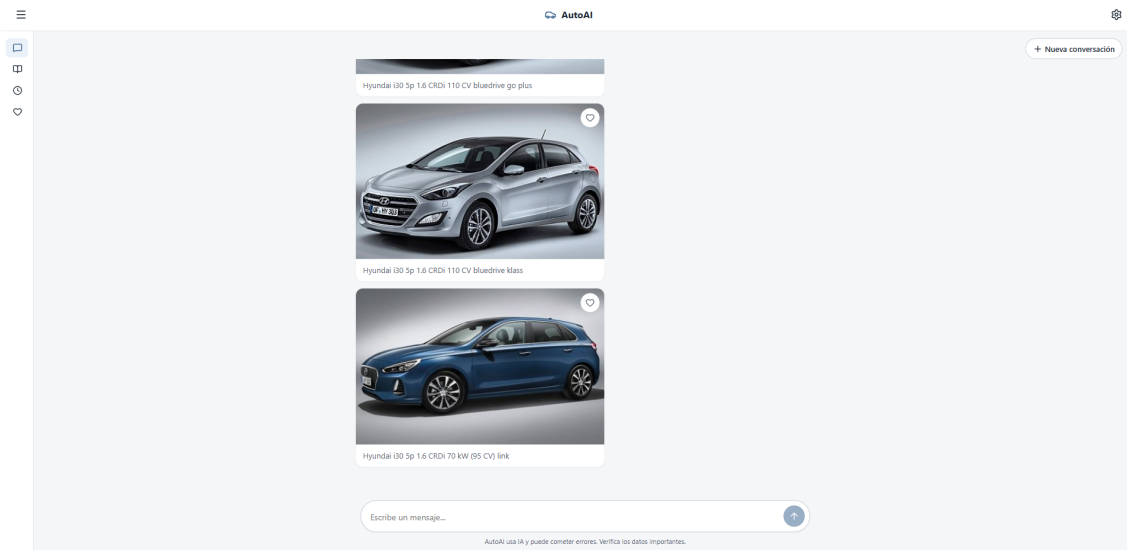


Figura 5.10: Interfaz conversacional con respuesta de imágenes.

Catálogo, filtros y comparador

El catálogo ofrece la vía de exploración estructurada. Presenta los vehículos en una rejilla de tarjetas y, sobre ella, un panel de filtros que se construye dinámicamente a partir de los metadatos que expone el *backend*: los desplegables de marca, combustible, carrocería, tracción y transmisión se pueblan con los valores realmente presentes en la base de datos, lo que evita ofrecer opciones sin resultados. Se complementan con campos para acotar rangos de precio, potencia y número de plazas, y con un buscador de modelo por texto. Los resultados se entregan paginados, y la vista contempla de forma explícita los estados de carga (mostrando una rejilla de marcadores de posición), de error (con opción de reintento) y de búsqueda sin resultados, de modo que la interfaz nunca queda en un estado ambiguo.

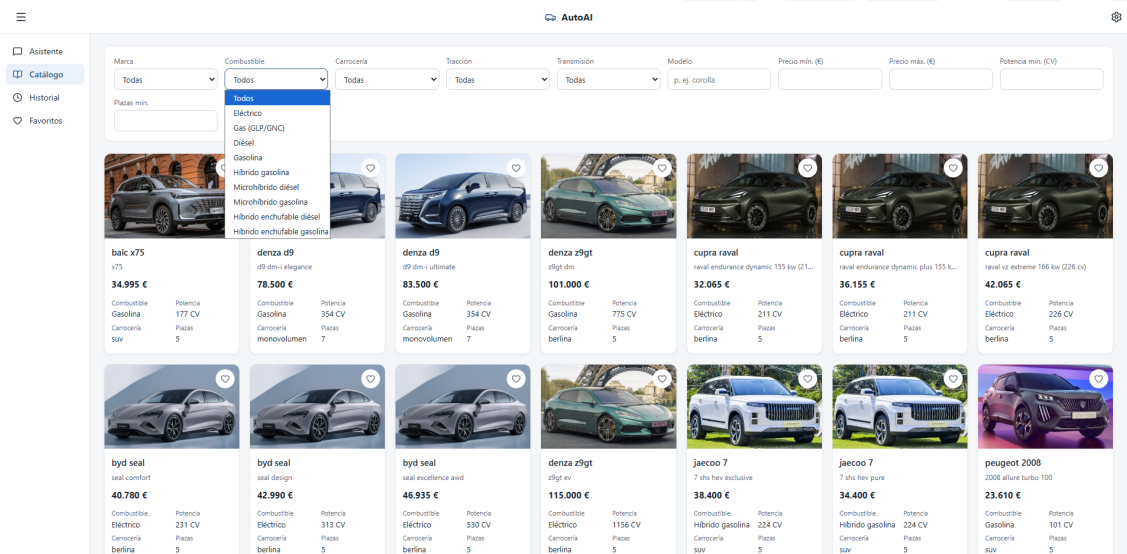


Figura 5.11: Vista de catálogo con filtros.

Favoritos e historial de consultas

Por último, el diseño contempla dos funcionalidades de personalización —**favoritos** e **historial**— con una decisión arquitectónica deliberada: ambas se resuelven íntegramente en el lado del cliente, persistiendo en el almacenamiento local del navegador, sin necesidad de cuentas de usuario ni de almacenamiento en el servidor. Esta opción es coherente con el alcance del proyecto —que no contempla la gestión de usuarios— y simplifica notablemente el *backend*, a costa de que la información quede ligada al navegador concreto.

Los favoritos permiten al usuario marcar vehículos de interés (desde las fichas con imagen que genera el asistente o desde el catálogo) y revisarlos después en una vista propia. El historial conserva las conversaciones mantenidas con el asistente, agrupadas en sesiones con un título derivado de la primera pregunta, de manera que el usuario pueda retomarlas o consultarlas más adelante. Para que estas vistas se mantengan siempre sincronizadas con los cambios —incluso entre varias pestañas abiertas simultáneamente—, la lectura del almacenamiento local se integra con el sistema de reactividad de la interfaz, de modo que cualquier modificación se refleja de inmediato en todas las vistas afectadas.

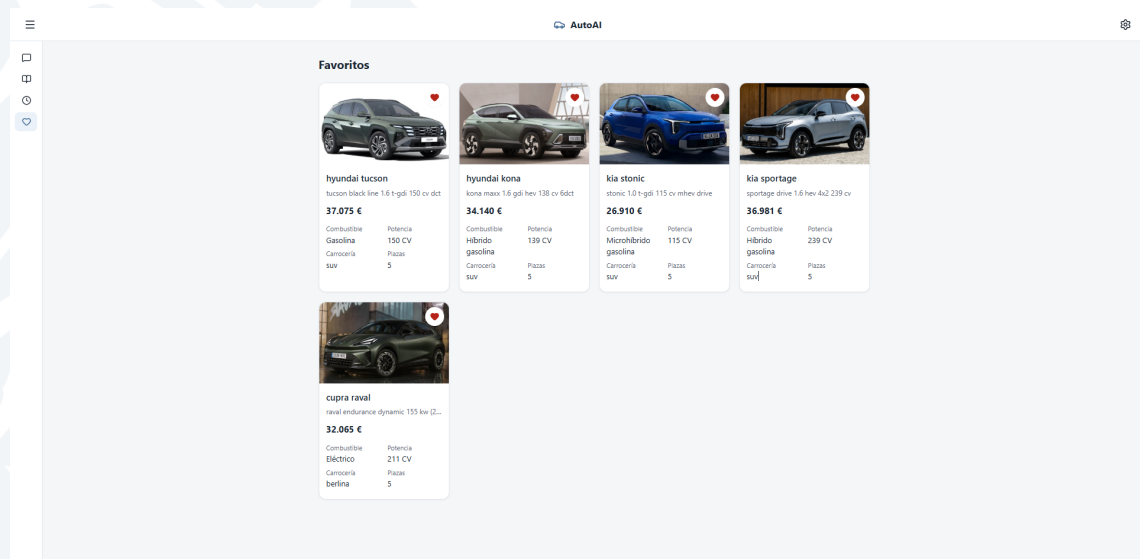


Figura 5.12: Vista de favoritos.

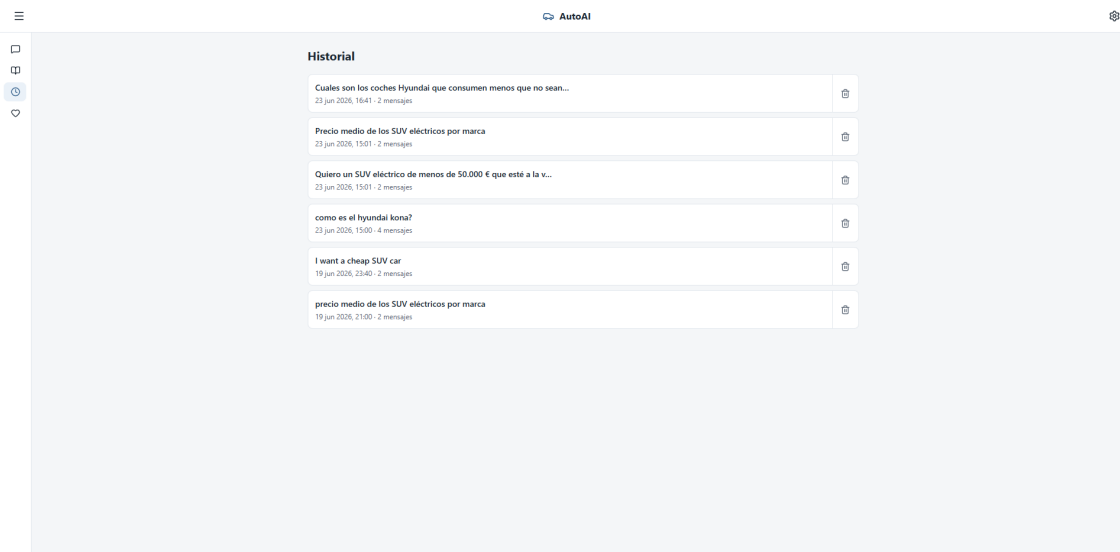


Figura 5.13: Vista de historial.

Preferencias de accesibilidad

La interfaz incorpora un panel de **preferencias de accesibilidad** que permite al usuario adaptar la presentación a sus necesidades. Se presenta como un cajón lateral (*drawer*) accesible —diseñado como diálogo modal con atrapado del foco, cierre con la tecla Esc y navegación completa por teclado— y reúne tres ajustes: el **tema** (claro u oscuro), el **tamaño de la fuente** (reducido, normal o ampliado) y el **idioma** de la interfaz (español o inglés). Coherentemente con la decisión adoptada para favoritos e historial, estas preferencias también se resuelven en el lado del cliente y persisten en el almacenamiento local del navegador, con unos valores por defecto definidos (tema claro, tamaño normal e idioma español) y una lectura defensiva que valida cada campo por separado y recurre al valor por defecto ante cualquier dato corrupto.

El diseño de este panel persigue que cada ajuste tenga un efecto global e inmediato sobre toda la aplicación, para lo cual cada preferencia se traduce en un cambio sobre el documento raíz. El tema alterna un atributo en la raíz del documento que conmuta la paleta de colores de toda la interfaz. El tamaño de fuente, dado que toda la tipografía está expresada en unidades relativas, se controla escalando un único valor en la raíz, lo que redimensiona la interfaz *proporcionalmente* sin romper la maquetación. El idioma, por su parte, ajusta tanto el atributo de idioma del documento como el sistema de internacionalización que traduce los textos de la interfaz; conviene recordar aquí que la localización solo afecta a los textos propios de la aplicación y a la prosa generada por el asistente, nunca a los datos de los vehículos.

Una última consideración de diseño atañe a la **experiencia de arranque**: para evitar el destello visual que se produciría si la aplicación se cargara primero con el tema por defecto y cambiara al tema preferido una vez iniciada, las preferencias guardadas se aplican al documento *antes* de que se monte la interfaz. Así, el usuario percibe desde el primer instante la presentación que había elegido.

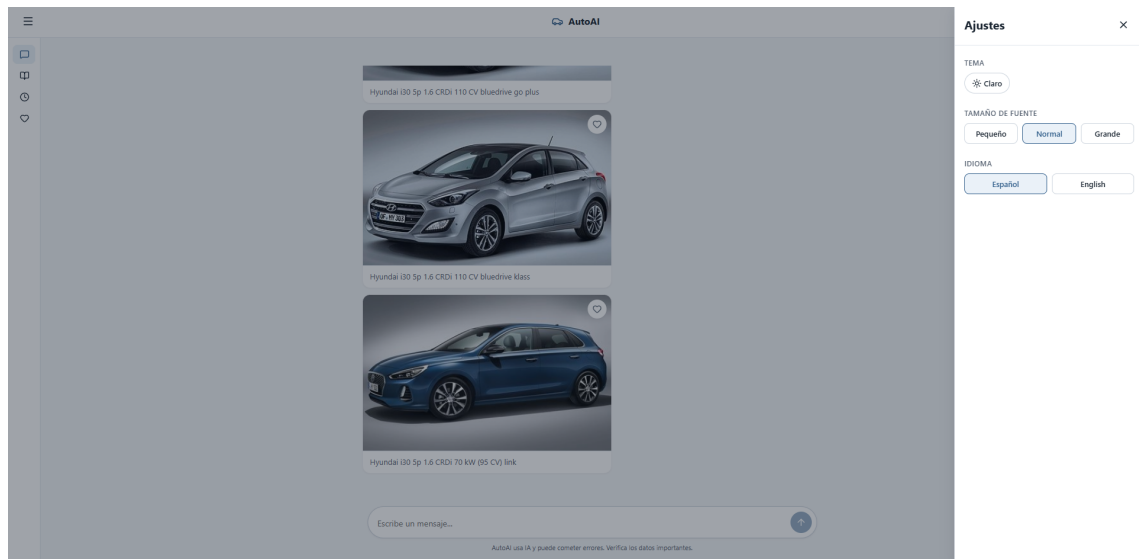


Figura 5.14: Vista del panel de preferencias de accesibilidad.

6 IMPLEMENTACIÓN

6.1 OBTENCIÓN DE DATOS: SCRAPING DE KM77.COM

6.1.1 Diseño del spider con Scrapy

6.1.2 Automatización periódica del scraping

6.2 PROCESO ETL Y CARGA EN LA BASE DE DATOS

6.3 BACKEND CON FASTAPI

6.3.1 Endpoints del catálogo y comparador

6.3.2 Modelos de datos y validación

6.4 INTEGRACIÓN DE LA INTELIGENCIA ARTIFICIAL

6.4.1 Traducción de lenguaje natural a filtros estructurados

6.4.2 Bucle de *tool-calling* sobre datos reales

6.4.3 Generación de respuestas explicadas

6.5 FRONTEND CON REACT

6.5.1 Interfaz conversacional

6.5.2 Catálogo, filtros y comparador

6.5.3 Favoritos e historial de consultas

6.6 CONTENERIZACIÓN Y DESPLIEGUE CON DOCKER

7 PRUEBAS Y EVALUACIÓN

7.1 ESTRATEGIA DE PRUEBAS

7.2 PRUEBAS DEL BACKEND Y DEL PIPELINE DE IA

7.3 EVALUACIÓN DE LA CALIDAD DE LOS RESULTADOS

7.4 DISCUSIÓN DE LOS RESULTADOS

8 CONCLUSIONES Y TRABAJO FUTURO

8.1 CUMPLIMIENTO DE LOS OBJETIVOS

8.2 CONCLUSIONES

8.3 LÍNEAS DE TRABAJO FUTURO

BIBLIOGRAFÍA

A CONTENIDO DE UNA ENTRADA EN .BIB

Nombre campo	Descripción
address	Usualmente la dirección de la editorial
author	Nombre(s) del (de los) autor(es)
title	Título del libro
chapter	El número de un capítulo (o sección, etc)
edition	La edición de un libro, por ejemplo,segunda
editor	Nombre(s) del (de los) editor(es)
howpublished	Forma en que fue publicada la obra
institution	Institución responsable de un informe técnico
journal	Nombre del periódico o revista
key	Empleado para la alfabetización, referencias cruzadas y para crear una clave cuando la información del autor no está disponible. No debe confundirse con la etiqueta usada en el cite y que debe colocarse al inicio de la entrada
month	El mes de publicación o, para un trabajo inédito, en el que fue escrito
note	Cualquier información adicional que pueda ayudar al lector
number	El número del periódico, la revista, el informe técnico o del trabajo en una serie
organization	La organización responsable de una conferencia o que publica un manual
pages	Números de páginas
publisher	El nombre de la editorial. No debe confundirse con el editor
school	Nombre de la escuela donde fue escrita una tesis
series	El nombre de una serie o conjunto de libros
title	El título del trabajo
type	El tipo de un informe técnico
volume	El volumen de un periódico o una revista, o de algún libro que conste de volúmenes
year	El año de publicación. Para un trabajo inédito, el año en que fue escrito. Generalmente debe consistir de cuatro dígitos, por ejemplo 200

Tabla A.1: Campos de una entrada en archivo de bibliografía .bib

B TIPOS DE REFERENCIAS

article	Un artículo de un periódico o revista
book	Un libro con una editorial que se indica en forma explícita. Los campos requeridos en este caso son author (autor), editor, title (título), publisher (editorial) y year (año)
booklet	Una obra que está impresa y encuadernada, pero sin una editorial o institución patrocinadora
conference	Lo mismo que inproceedings, incluido para compatibilidad con el lenguaje de marcación Scribe
inbook	Una parte de un libro, que puede ser un capítulo (o sección) o un rango de páginas
incollection	Una parte de un libro que tiene su propio título
inproceedings	Un artículo en las actas de sesiones (proceedings) de una conferencia
manual	Documentación técnica
mastersthesis	Una tesis de maestría (Master thesis) o proyecto fin de carrera
misc	Para uso cuando los demás tipos no corresponden
phdthesis	Una tesis de doctorado (Ph D thesis)
proceedings	Las actas de sesiones de una conferencia
techreport	Un reporte publicado por una escuela u otra institución, usualmente numerado dentro de una serie
unpublished	Un documento que tiene un autor y título, pero que no fue formalmente publicado

Tabla B.1: Tipos posibles de elementos en bibliografía con natbib

ARTEFACTOS SOFTWARE

En este archivo se incluirá tanto el resumen en castellano como su traducción al inglés. Los dos párrafos estarán ligeramente separados.

El propósito del trabajo en una o dos frases. El diseño y metodología utilizada, los resultados más significativos del trabajo realizado y un breve resumen de las conclusiones. Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG.

Put here the english translation Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG. Los dos párrafos estarán ligeramente separados.

